

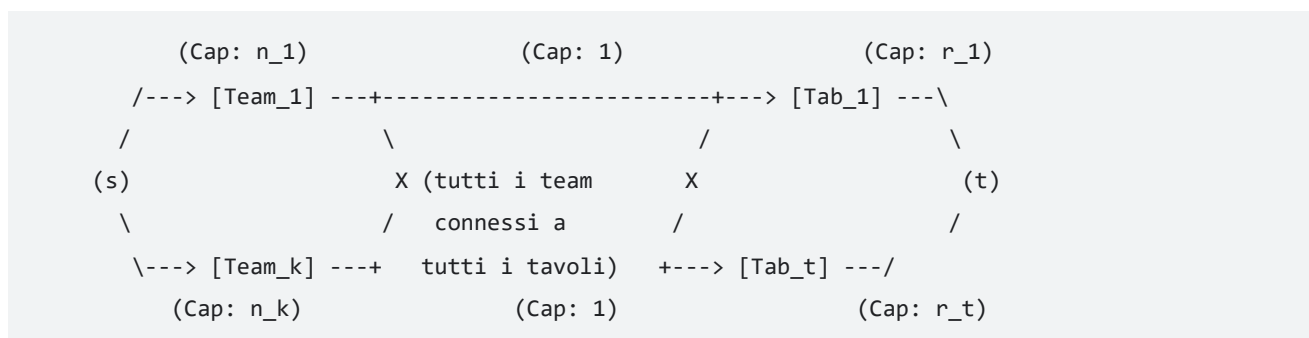
Exam 12-02-2025 - Selected Solutions

Problem 1: Seating Arrangement Problem (Max-Flow)

1. Modello Max-Flow e Condizione di Successo

Costruzione del Grafo: Costruiamo una rete di flusso orientata $G = (V, E)$ per modellare le assegnazioni dei dipendenti ai tavoli: * **Nodi:** * Una sorgente s e un pozzo t . * Un nodo $Team_i$ per ciascun team, con $i = 1, \dots, k$ [cite: 1]. * Un nodo Tab_j per ciascun tavolo, con $j = 1, \dots, t$ [cite: 1]. * **Archi e Capacità:** * **Sorgente $\rightarrow$ Team:** Aggiungiamo un arco $(s, Team_i)$ con capacità $c = n_i$ per ogni team. Questo limita il numero di persone che possono "uscire" dal team al numero di membri iscritti [cite: 1]. * **Team $\rightarrow$ Tavoli:** Aggiungiamo un arco $(Team_i, Tab_j)$ da ogni team a ogni tavolo con capacità $c = 1$. Questo è il vincolo cruciale: garantisce che al massimo un singolo membro del team i possa essere assegnato al tavolo j , rispettando la regola che dipendenti dello stesso team non siano allo stesso tavolo [cite: 1]. * **Tavoli $\rightarrow$ Pozzo:** Aggiungiamo un arco (Tab_j, t) con capacità $c = r_j$ per ogni tavolo. Questo vincola il numero massimo di persone assegnabili a quel tavolo in base ai suoi posti disponibili r_j [cite: 1].

Schema del Grafo:



Condizione di Successo: Il problema decisionale restituisce `true` (è possibile far sedere tutti rispettando i vincoli) se e solo se l'algoritmo di Max-Flow trova un flusso massimo f^* tale che $f^* = \sum_{i=1}^k n_i$. Ovvero, tutto il flusso originato dai dipendenti riesce a raggiungere i tavoli e infine il pozzo.

2. Complessità Computazionale

Definiamo le dimensioni della rete in funzione dei parametri dati: * **Numero di vertici $|V|$:** 1 (sorgente) + k (team) + t (tavoli) + 1 (pozzo) = $O(k + t)$. * **Numero di archi $|E|$:** k (sorgente-team) + $k \cdot t$ (team-tavolo) + t (tavolo-pozzo) = $O(k \cdot t)$. * **Valore del Flusso Massimo F^* :** $\sum_{i=1}^k n_i$.

Se applichiamo l'algoritmo di **Ford-Fulkerson**, il cui tempo di esecuzione standard è limitato da $O(|E| \cdot F^*)$, il runtime per risolvere il problema sarà [cite: 1]:

$$O\left(k \cdot t \cdot \sum_{i=1}^k n_i\right)$$

Problem 2: Max-Weight Independent Set on Line Graph

(a) Algoritmo e Ricorrenza Sia $OPT(i)$ il peso massimo ottenibile considerando i primi i nodi di un grafo a linea $G = (V, E)$. Al nodo i , abbiamo due scelte possibili: 1. **Se non lo prendiamo:** Il peso massimo è semplicemente quello ottenuto sui primi $i - 1$ nodi, ovvero $OPT(i - 1)$. 2. **Se lo prendiamo:** Poiché per definizione di Independent Set non possiamo prendere nodi adiacenti, il nodo $i - 1$ viene escluso, e possiamo sommare il peso w_i al peso ottimo calcolato fino al nodo $i - 2$, ovvero $w_i + OPT(i - 2)$.

La ricorrenza ottimale è dunque:

$$OPT(i) = \max(w_i + OPT(i - 2), OPT(i - 1))$$

Con i casi base: $OPT(0) = 0$ e $OPT(1) = w_1$.

(b) Pseudo-codice

```
Algorithm MaxWeightIndependentSet(w, n):
    If n == 0:
        return 0
    If n == 1:
        return w[1]

    OPT = array of size n+1
    OPT[0] = 0
    OPT[1] = w[1]

    For i from 2 to n:
        OPT[i] = max(w[i] + OPT[i-2], OPT[i-1])

    Return OPT[n]
```

(c) Analisi della Complessità * Tempo: L'algoritmo utilizza un unico ciclo `For` che itera $n - 1$ volte. All'interno del ciclo vengono eseguite solo operazioni elementari (somme e confronti) in tempo $O(1)$. La complessità temporale totale è quindi $O(n)$ [cite: 4]. * **Spazio:** Viene utilizzato un array di supporto `OPT` di dimensione $n + 1$, portando la complessità spaziale a $O(n)$ [cite: 4] (ottimizzabile a $O(1)$ memorizzando solo le ultime due variabili).

Problem 3: Max-Cut Problem

1. Analisi dell'algoritmo randomizzato ($\frac{1}{2}$ -approssimazione): Assegnando casualmente ogni vertice a uno dei due insiemi S o $V \setminus S$ con probabilità $\frac{1}{2}$ [cite: 4], consideriamo un singolo arco $e = (u, v)$. La probabilità che i suoi due estremi finiscano in insiemi diversi (e che quindi l'arco venga tagliato) è esattamente $\frac{1}{2}$. Usando la linearità del valore atteso, il numero atteso di archi tagliati è $\sum_{e \in E} \frac{1}{2} = \frac{|E|}{2}$. Poiché il valore ottimo OPT non può superare il numero totale di archi $|E|$, il valore atteso è $E[X] \geq \frac{OPT}{2}$, garantendo un'approssimazione in media di $\frac{1}{2}$ [cite: 4].

2. NP-Hardness (Bonus - Riduzione da NAE-3SAT): Per dimostrare la NP-hard del Max-Cut [cite: 4], si riduce il problema NAE-3SAT. Per ogni variabile si crea un arco di coerenza, e per ogni clausola si costruisce un "gadget" a triangolo con tre archi. Selezionando un taglio massimo si forza l'assegnazione di verità a soddisfare la condizione Not-All-Equal delle clausole, poiché un triangolo può tagliare al massimo 2 dei suoi 3 archi.

3. Analisi della Complessità: Il lancio di una moneta per ciascuno dei n vertici e la successiva partizione richiedono un tempo lineare rispetto al numero di vertici e archi, risultando in una complessità temporale di $O(|V| + |E|)$.

Problem 4: Subset-Sum is NP-Complete

1. Appartenenza a NP: Dato un sottoinsieme $S \subseteq X$ come certificato, un verificatore può sommare i numeri in tempo polinomiale e controllare se la somma è esattamente pari a T . Dunque $\text{Subset-Sum} \in \text{NP}$ [cite: 4].

2. NP-Hardness (Riduzione diretta da 3D-Matching): Sia data un'istanza di 3D-Matching con tre insiemi disgiunti X, Y, Z di dimensione n e un insieme di m terne M . * **Costruzione dei numeri:** Scegliamo una base numerica $b = m + 1$ per evitare i riporti. Per ogni terna $(x_i, y_j, z_k) \in M$, creiamo un numero intero di $3n$ cifre (in base b) che vale 1 nelle posizioni $i, n + j$ e $2n + k$, e 0 altrove. * **Target (T):** Impostiamo un numero di $3n$ cifre tutte uguali a 1 (in base b). * **Dimostrazione:** Se esiste un 3D-Matching perfetto, la somma dei numeri corrispondenti raggiunge esattamente T senza riporti, poiché ogni elemento viene coperto una sola volta. Viceversa, se il Subset-Sum trova una soluzione, l'assenza di riporti (garantita dalla base $m + 1$) assicura che ogni posizione sia coperta esattamente da una terna, dando un 3D-Matching valido.